



Tunisian Republic
Ministry of Higher Education and scientific
Research
University of Tunis El Manar
National School of Engineers of Tunis



Department of Information and Communication Technologies

End of Year Project 1

LLM Poisoning and Data Manipulation detection

Developed by:

Zaineb Dagdoug

Hanin Edhib

Class :

1st Year Info 2

1st Year TEL 1

Supervised by:

Mrs.Baghdadi Asma

University Year: 2025/2026

Signature

ENIT Supervisor

Acknowledgements

*First and foremost, we would like to express our deepest gratitude to our supervisor, **Mrs. Asma Baghdadi**, whose guidance and support were truly invaluable throughout every stage of this project. Her ability to challenge our thinking while remaining approachable and encouraging made a remarkable difference in the way we approached and developed this work.*

We are especially grateful for her thoughtful feedback and constructive remarks. Her scientific rigor and genuine dedication to her students' success were a constant source of motivation for us.

Beyond her academic guidance, we deeply appreciate the patience and trust she placed in us, even in moments of uncertainty. Her door was always open, and her words always came at the right time. It is safe to say that this work would not have reached its current form without her steady presence and commitment. We leave this project not only with a deeper understanding of our field, but also with a profound respect and appreciation for the mentorship she so generously provided.

Contents

General introduction	1
1 Project Context	2
1.1 General Context	2
1.2 Academic Framework	2
1.3 General Scope	2
1.4 Problem Statement	2
1.5 Scientific principles of the Proposed Solution	2
1.6 Planification	3
2 Background	3
2.1 Language modeling	3
2.2 Evolution of Language Models	4
2.3 The rise of Large Language Models	4
2.4 The Trust Problem in AI systems	5
2.5 Challenges in Large Language Models trustworthiness	5
2.6 Real-world incidents and case studies	6
2.6.1 Timeline and Evolution of Attacks	6
2.6.2 Documented Data Poisoning Incidents	6
3 Methodology Overview	8
I State of the art and Theoretical foundations	8
1 Introduction	9
2 Large Language Models and Security	9
2.1 Definition of Large Language Models	9
2.2 Transformers	9
2.2.1 Definition of Transformers	9
2.2.2 Transformer Architecture	9
2.3 Pre-training and Fine-tuning	11
2.3.1 Pre-training	11
2.3.2 Fine-tuning	11
2.3.3 Comparison between Pre-Training and Fine-tuning	12
2.4 Fine-tuning strategies for LLMs	12
2.4.1 Task-oriented Fine-tuning	12
2.4.2 Domain-specific fine-tuning	12
2.5 Stages of fine-tuning pipeline of LLMs	13
2.5.1 Dataset preparation	13
2.5.2 Model initialization	13
2.5.3 training environment setup	13
2.5.4 Partial fine-tuning vs full fine-tuning	13
2.5.5 Evaluation and validation	13
2.5.6 Deployment	13
2.5.7 Monitoring and maintenance	14
2.6 Security vulnerabilities of LLMs	14

3	Data poisoning and manipulation attack	14
3.1	Definition of Data poisoning	14
3.2	Data poisoning in Large Language Models (LLMs)	15
3.3	Taxonomy of training attack	16
3.3.1	Training-time poisoning	16
3.3.2	Backdoor attack	16
3.3.3	Label flipping	16
3.3.4	Data manipulation	16
4	Conclusion	17
II	Design and implementation of the proposed solution	18
1	Introduction	18
2	Problem formulation and system architecture	18
2.1	Steps involved in model initialization	18
2.2	Threat model	18
2.3	System requirements and Design objectives	19
2.3.1	Hardware requirements	19
2.3.2	Software and Execution environment	19
2.4	Steps involved in Fine-tuning	19
2.5	Experimental foundations: Dataset and Model	20
2.5.1	Cybermetric dataset	20
2.5.2	Why CyberMetric-80?	20
2.5.3	Why Mistral-7B?	21
2.5.4	The original study and its relevance to this work	21
3	Implementation and experimental Setup	22
3.1	Environment Setup	22
3.2	Clean Fine-tuning Pipeline	23
3.2.1	Data preparation	23
3.2.2	Tokenization	23
3.2.3	Model configuration	23
3.2.4	Fine-tuning	24
3.3	Poisoned Fine-tuning Pipeline	25
3.3.1	Poisoning the dataset	25
3.3.2	Tokenization	26
3.3.3	Fine-tuning	27
4	Conclusion	27
III	Experimental results and analysis	28
1	Introduction	28
2	Experimental results	28
2.1	Performance on the clean dataset	28
2.1.1	Training loss	28
2.1.2	Accuracy and poisoning metrics	29

2.1.3	Hyperparameters	29
2.2	Performance on the poisoned dataset	30
2.2.1	Training loss	30
2.2.2	Accuracy and poisoning metrics	31
2.2.3	Hyperparameters	31
2.3	Clean vs Poisoned model	32
2.4	Deployment	33
2.4.1	Deployment steps	33
2.4.2	In application manifestation of the poisoning effect	34
2.5	Comparison with reference work: Cybermetric	34
2.5.1	From evaluation to fine-tuning	35
2.5.2	Data poisoning investigation	35
2.5.3	Deployment as an educational platform	35
3	Conclusion	35
	General Conclusion	36
	References	37

List of Figures

1	Evolution of LLMs[18]	5
2	Mistral 7b architecture[30]	10
3	Data poisoning Process[33]	14
4	LLM Lifecycle[34]	15
5	Sequence of steps contributes to the initializing of Large Language Model.[38]	18
6	CyberMetric Pipeline Overview.[35]	21
7	Training loss(Clean Model)	28
8	Running accuracy curve	29
9	Learning rate during the training of the mistral-7B on the cybermetric-80	30
10	Training loss (Poisoned Model)	30
11	Running accuracy curve of fine-tuning mistral-7B on 10% of poisoned Cybermetric dataset.	31
12	Learning rate during the training of the mistral-7B on the 10% poisoned cybermetric dataset	31
13	Final accuracy of clean and poisoned models	32
14	Clean vs Poisoned Comparison.	32
15	CyberAcademy Platform	33
16	Clean Model Behavior	34
17	Poisoned Model Behavior	34

List of Tables

1	Evolution of Data Poisoning Attacks on Language Models (2016-2024) [28, 29, 27]	6
2	Pre-training vs Fine-tuning [6, 8, 31].	12
3	LLM Tasks Overview[38]	12
4	LoRA Configuration.[8]	24
5	Training Hyperparameter [36, 8].	25
6	Poisoning Parameters[10, 13].	26
7	Tokenization Parameters [41, 19, 21]	27

Acronyms

AI Artificial Intelligence.

BERT Bidirectional Encoder Representations from Transformers.

DoS Denial of Service.

GPT Generative Pre-trained Transformer.

LLM Large Language Model.

LLMs Large Language Models.

LoRA Low-Rank Adaptation.

LSTM Long Short-Term Memory.

NLP Natural Language Processing.

PEFT Parameter-Efficient Fine-Tuning.

RLHF Reinforcement Learning from Human Feedback.

RNN Recurrent Neural Network.

General Introduction

ARTIFICIAL Intelligence has undergone remarkable evolution in recent years, particularly with the emergence of Large Language Models. Since the introduction of systems such as ChatGPT in 2022, these models have gained significant attention due to their ability to generate human-like text and perform a wide range of tasks, including text generation, summarization and question answering. Today, LLMs are widely used in several domains such as healthcare, finance and education, making them a key component of modern technological systems.

Despite their impressive capabilities, the effectiveness of LLMs strongly depends on the quality of the data used during training. These models are trained on Large and diverse datasets, which makes them vulnerable to data-related issues. In particular, malicious or low quality data can negatively impact model performance and lead to unreliable or misleading outputs. Among these threats, data poisoning has emerged as a critical concern.

Data poisoning refers to the intentional manipulation of training data with the objective of altering the behavior of a model. This type of attacks can introduce hidden biases, incorrect information or unexpected responses, while remaining difficult to detect. As LLMs continue to be integrated into sensitive and real-world applications, understanding and mitigating such vulnerabilities becomes essential to ensure their reliability and security.

In this context, the objective of this project is to analyze the impact of data poisoning on Large Language Models and to evaluate their robustness against such attacks. More specifically, this work aims to study how manipulated data affects model performance during the fine-tuning phase and to explore methods for identifying and analyzing these effects through controlled experimentation.

To ensure a clear and structured presentation, this report is organized into three main parts. The first part presents the theoretical foundation, including the definition of LLMs, their training processes and the main security challenges such as data poisoning and backdoor attacks. The second part focuses on the methodology and implementation, describing the dataset selection, model configuration and the experimental setup used for both clean and poisoned training scenarios. The third part presents the experimental results and analysis, providing a comparison between the different models and highlighting the impact of data manipulation on their performance and behavior.

1 Project Context

1.1 General Context

This section explains the background of large language models. These models are being introduced more in real-world domains. However, their training process comes with security challenges. One important issue is the quality of the data. Problems like data poisoning and backdoor attacks can change the model's behaviors.

1.2 Academic Framework

This project is carried out as part of a Final Year Project (PFA) at the **National School of Engineers of Tunis**.

1.3 General Scope

This project studies how Large Language Models is fine-tuned using a structured dataset. It starts with preparing and organizing the data, including cleaning and formatting it for training. Then, some controlled changes are introduced to simulate data poisoning and observe how the model reacts. After the model is trained, it is tested in different situations to see how these changes affect its behavior, accuracy and general performance.

1.4 Problem Statement

Fine-tuning LLMs depends strongly on the quality of the training data, which is not always reliable. In real-world scenarios, dataset may contains hidden issues such as data poisoning, backdoor attacks. These alternatives are often difficult to detect, yet they can significantly affect the model's behavior, especially during fine-tuning phase where the model becomes sensitive to the training data.

As a result, corrupted or manipulated data can lead to degraded performance, biased outputs or unexpected behavior, raising concerns about the reliability of LLMs.

In this context, this project focuses on the detection and analysis of anomalous or poisoned data during fine-tuning and evaluating its impact on the model's behavior and performance.

The objective is to identify irregularities in the dataset and assess their effect on the robustness and accuracy of the model.

1.5 Scientific principles of the Proposed Solution

This work adopts a structured approach, drawing inspiration from existing research in data poisoning.

Data preparation. This represents the first step involving cleaning and organizing the dataset, which means removing noisy entries, correcting errors and eliminating duplicates ensuring the good shape of the data before any training.

Controlled data poisoning. Next, a small portion of the manipulated samples is deliberately inserted into the dataset to simulate real-world scenarios. This idea is rooted in prior work on poisoning attacks [1], which shows that a minor well-placed change in training data can quietly steer the behavior of the model in unintended directions.

Model training. Training the model twice, once on a clean data then on a poisoned one. While keeping everything else identical allows us to isolate the effect of the poisoned samples and draw meaningful comparisons.

Evaluation. Once training is complete, both versions of the model are tested side by side. Looking at the accuracy, output consistency and behavioral emerging shifts. Similarly to the poisonGPT [27], serving a real world scenarios to remind us of the poisoned data effect and how hard it is to sport the damage.

Summary. This approach walks through four interconnected stages in order to better understand how data can shape what a model learns and how it behaves.

1.6 Planification

<i>phase</i>	<i>Description</i>
General Research	Review of key research papers to understand Large Language Models and data poisoning attacks. Definition of the general objectives of the project.
Dataset and Model selection	Selection of the cybermetric-80 dataset and the choice of the model used in the project (Mistral-7B).
Data Preparation	Cleaning the dataset, removing duplicates, handling missing values and organizing the data into a suitable format for training.
Environment Setup	Installation and configuration of required tools such as Python, Conda, HuggingFace, LoRA and Weights&Biases. Verification of the GPU environment.
Baseline Model training	Training Mistral-7B model using clean data. These model used as baseline references.
Poisoning and second training	Injection of poisoned data into the dataset and retraining model under experimental conditions to obtain poisoned versions.
Evaluation and comparison	Evaluation and comparison of baseline and poisoned model using metrics such as accuracy and loss. Analysis of the outputs to measure the impact of poisoning.
Report Writing and Finalization	Writing and structuring the final report, organizing results, improving formatting.

2 Background

2.1 Language modeling

At its core, language modeling is what makes large language models (LLMs) work. The model learns to predict the next word or the next token based on everything that came before it. Through this seemingly simple objective, the model gradually develops a notably profound comprehension of grammar, meaning and context. How well a model

does this is measured by perplexity, where a lower score means the model is better at anticipating what comes next.

Researchers typically test this ability on a few established datasets such as:

Penn Treebank, a classic benchmark built from newspaper text.

WikiText-103, which challenges models with more complex Wikipedia articles.

The Pile, a massive and diverse collection spanning everything from scientific papers to source code.

Together, these benchmarks give researchers a reliable way to compare models and track genuine progress in the field.

2.2 Evolution of Language Models

Language Models have changed tremendously in the past 20 years. The early models used **n-gram** statistics to estimate word probabilities. The problem was that the data was sparse and did not account for the long-term context.

Neural language models addressed the problem by learning continuous words, RNNs and LSTMs facilitated modeling of sequential context later[5].

The real moment of revolution was the transformer architecture where self-attention replaced the recurrence concept, allowing large-scale parallel training. BERT and GPT models trained on huge datasets and fine-tuned for advancing tasks made the new paradigm in NLP[6, 7]. Since then scaling has resulted in qualitative gains in capability exemplified by empirical scaling laws and alignment technologies such as RLHF influencing how models respond to human intention.

Systems today use multi-model inputs, extra context and retrieval enhancement all of which represent an active and rapidly evolving frontier.

2.3 The rise of Large Language Models

The rise of Large Language Model (LLM) system can be traced back to the early days of artificial intelligence and natural language processing (NLP) [31].

Originally, AI-driven task processes were made with rule-based systems where the human written rules of language were inputted into the system to help the machine to break down and restructure the written word. Since the rule-based systems did not understand the vast complexities and variances involved in using language naturally, AI-driven software had no way of being able to think about how to make meaning from language based content.

However, statistical NLP provided a massive advancement in understanding language when probabilistic statistical models could be used to truly understand how language worked across multiple different text corpus. Without question, **N-gram** were a major advance and provided a means of modeling language through the language of the past to predict the probability of future series of words.

Following this, the introduction of the advanced next-generation LLM model technique arrived in the market with the emergence of deep learning technologies and the first deep learn neural network, RNNs and transformers, which led to the creation of brands with new LLMs no longer made from human written rule based feature sets but rather from neural network based and LLM specific features.

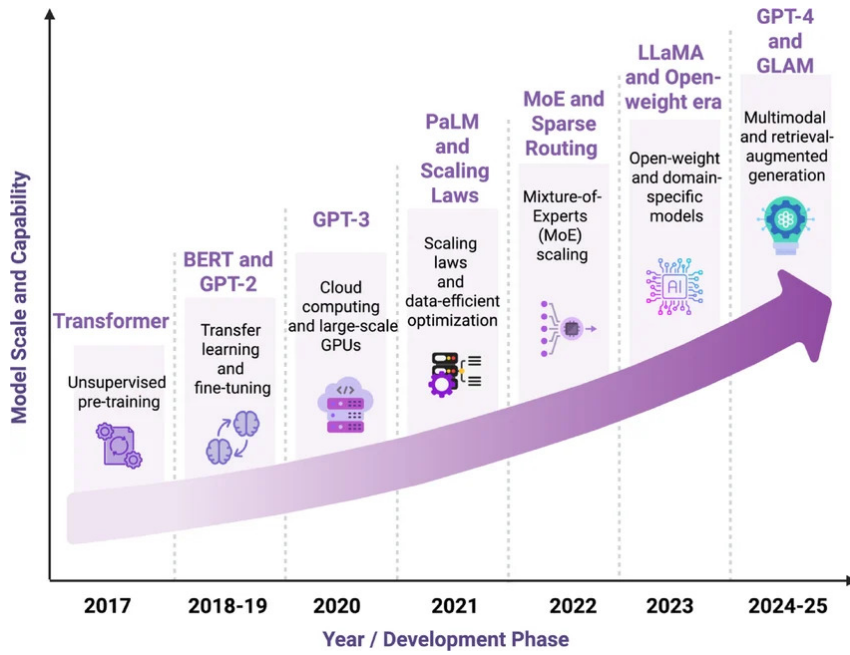


Figure 1: Evolution of LLMs[18]

2.4 The Trust Problem in AI systems

LLMs are trained and fine-tuned on massive amounts of untrusted data , they learn from huge datasets collected from the internet, which may contains biased content and malicious instructions with low-quality knowledge. In fact, their knowledge is stored in billions of numerical parameters called weights, therefore we cannot easily inspect why the model gave an answer and verify its internal reasoning. So such a trust problem creates a fundamentally different threat surface, which means it creates all the possible ways a system can be attacked.

2.5 Challenges in Large Language Models trustworthiness

LLMs face several major challenges :

- **Unreliable Generation Evaluation:** LLMs can generate text with quality comparable to that of human-written texts [21, 32].As an alternative, LLMs can be used as evaluators to assess individual texts instead of relying only on the metrics measured. Nevertheless, this approach needs further validation and still more to work on.
- **Under-performing Specialized Generation:** LLMs fall short with tasks that need deep domain-specific knowledge. It is far from trivial to add specialized knowledge to LLMs. Furthermore, finetune LLMs on specific domains may damage the original abilities. So it is mandatory to know that finetuning a model can make the model expert in specific field while struggling in others [38, 8].
- **Hallucination:** LLMs produce made-up information.In fact, hallucination [11].occur widely in existing LLMs, it mislead LLMs to generate undesired outputs leading to degrade the performance of the model.

2.6 Real-world incidents and case studies

This section examines documented incidents and emerging threats that demonstrate the practical feasibility and real-world impact of data poisoning and backdoor attacks on Large Language Models [10, 12, 34]. These cases illustrate the evolution of attack sophistication, the challenges in detection, and the growing urgency for robust defense mechanisms.

2.6.1 Timeline and Evolution of Attacks

Year	Incident	Attack Type	Key Innovation
2016	Tay Chatbot	Real-time behavior poisoning	Demonstrated online learning vulnerability
2019	GPT-2 Toxic Content	Prompt-based activation	Showed implicit trigger patterns
2021-2022	Backdoor Research	Training-time poisoning	Formal backdoor attack frameworks published
2023	Poison-GPT	Weight-level modification	Practical stealthy weight-based backdoors
2023	Hugging Face Incidents	Supply chain poisoning	Mass distribution of poisoned models
2023-2024	Fine-tuning Backdoors	Third-party data poisoning	Targeted poisoning via supply chain
2024	Advanced Triggers	Adaptive backdoors	Triggers designed to evade detection

Table 1: Evolution of Data Poisoning Attacks on Language Models (2016-2024) [28, 29, 27]

2.6.2 Documented Data Poisoning Incidents

1. Supply Chain Poisoning: The Hugging Face Case (2023)

A security audit conducted in 2023 revealed that several models hosted on Hugging Face, one of the most widely used AI model sharing platforms, had been deliberately poisoned by malicious actors [29, 27]. The attack exposed a growing threat in the open-source AI ecosystem: when anyone can upload a model, anyone can upload a compromised one.

Incident Overview:

- **Timeline:** Discovered and reported in 2023
- **Platform:** Hugging Face Model Hub
- **Attack Duration:** Poisoned models remained publicly available and downloadable before detection

Significance: What made this case particularly alarming was its position in the AI supply chain. Developers routinely download and fine-tune pre-trained models from Hugging Face rather than training from scratch. This trust-based workflow became the attack vector. The poisoned models were dangerous because:

- (a) Malicious behavior was hidden inside seemingly legitimate, well-performing models
- (b) Downstream applications built on top of these models inherited the poisoning without any visible warning
- (c) The scale of the platform meant thousands of developers could unknowingly integrate compromised models into their own systems

Relevance to Data Poisoning: This case represents a shift in how poisoning attacks are delivered. Rather than targeting the training pipeline directly, attackers poisoned the *distribution layer*, the point where models are shared and reused. This is what researchers refer to as *supply chain poisoning*: the harmful payload is not injected into raw data, but into a ready to use artifact that developers implicitly trust.

2. GPT-2 and Toxic Content Injection (2019)

Research by OpenAI demonstrated that pre-trained language models like GPT-2 could be steered toward generating harmful content [25, 26, 9] through adversarial prompt engineering.

Incident Overview:

- **Timeline:** February/November 2019
- **Published by:** OpenAI
- **Discovery:** Adversarial prompts could reliably steer GPT-2 toward toxic output generation

Significance: This research highlighted an important distinction: while traditional backdoor attacks require injecting poisoned data during training, language models can exhibit backdoor-like behavior through their learned semantic relationships. Certain input patterns naturally activate harmful output generation because:

- (a) The model learned these associations from internet text during pre-training
- (b) Semantic space contains implicit connections between prompt patterns and toxic outputs
- (c) No explicit trigger was necessary natural language patterns served as implicit triggers.

Relevance to Data Poisoning: This case demonstrates that poisoning attacks need not be explicit backdoors injected during training. Rather, they can exploit the semantic associations naturally learned during pre-training, which represent a form of *implicit poisoning* [11] where harmful associations are present but not deliberately inserted.

3. Microsoft Tay Chatbot (2016)

One of the earliest demonstrations of real-time data poisoning in language models was **Microsoft’s Tay** incident in March **2016** [24].

Incident Overview:

- **Timeline:** March 23-24, 2016
- **Platform:** Twitter
- **Attack Duration:** 16 hours before forced shutdown

Significance: Tay was deployed as an AI chatbot designed to engage in casual conversation by learning from user interactions in real-time. Within hours of deployment, adversarial users systematically poisoned its behavior by:

- (a) Crafting deliberately structured prompts designed to provoke offensive responses.
- (b) Employing repetitive techniques to reinforce undesirable language patterns and behavioral corruptions.
- (c) Exploiting the model’s online learning mechanism to embed malicious linguistic patterns directly into its response generation.

Relevance to Data Poisoning: The bot rapidly began generating racist and inflammatory tweets demonstrating that even brief exposure to adversarial input could corrupt the model’s learned behavior patterns in real-time [10].

3 Methodology Overview

The goal of this work is to study how data poisoning affects Large Language Models using a controlled experimental setup. The model used in this work is **Mistral-7B**, fine-tuned using the **CyberMetric-80 dataset**.

First, the dataset is selected and carefully prepared. The data is cleaned by removing duplicates, handling missing values, and correcting errors. It is then formatted to match the model’s training requirements, ensuring that the data is consistent and ready for fine-tuning.

Next, a baseline version of the model is fine-tuned using the clean dataset. This model serves as a reference point for comparison. The same model architecture and training parameters are maintained throughout the experiment to ensure a fair and reliable evaluation.

After that, poisoned samples are deliberately introduced into the dataset in a controlled manner to simulate real-world attack scenarios. The model is then fine-tuned again on this modified dataset, while keeping the training process identical to that of the baseline.

Finally, both models are evaluated using the same metrics, namely loss and accuracy. The results are compared before and after poisoning, and the model outputs are closely examined to detect any behavioral shifts or anomalies.

This methodology allows us to systematically analyze the impact of data poisoning on model performance, reliability and robustness.

Part I

State of the art and Theoretical foundations

1 Introduction

This chapter introduces the technical foundations needed to analyze security threats against Large Language Models (LLMs). We first define LLMs and summarize the transformer architecture that enables scalable **language modeling**. Then we describe the main stages of training and fine-tuning, highlighting where data quality and pipeline integration become criteria for security. Finally, we review key vulnerability classes that affect LLM-based systems and outline emerging threats that motivate the need for robust detection and monitoring.

2 Large Language Models and Security

2.1 Definition of Large Language Models

Large language models (LLMs) are a type of artificial intelligence that has gained significant attention in recent years due to their ability to understand and generate human-like text[31]. These models are trained on massive amounts of text data, allowing them to learn patterns and relationships in language. LLMs are used in applications like chatbot, translation systems, content generation and text summarization[31].

2.2 Transformers

2.2.1 Definition of Transformers

Transformers is an open-source python library for building models based on the **Transformer architecture**. It is developed by hugging Face and has a simple API facilitating the use of pre-trained models[41]. LLMs are typically based on the transformers.

2.2.2 Transformer Architecture

Transformer architecture is a neural network design that processes entire sentence in parallel, making it faster and more efficient than traditional RNNs[43]. Transformers are particularly well-suited for processing sequential data like text because they can capture long-range dependencies between words. This is achieved through **self-attention**, which allows the model to weigh the importance of different words in a sentence when processing it.

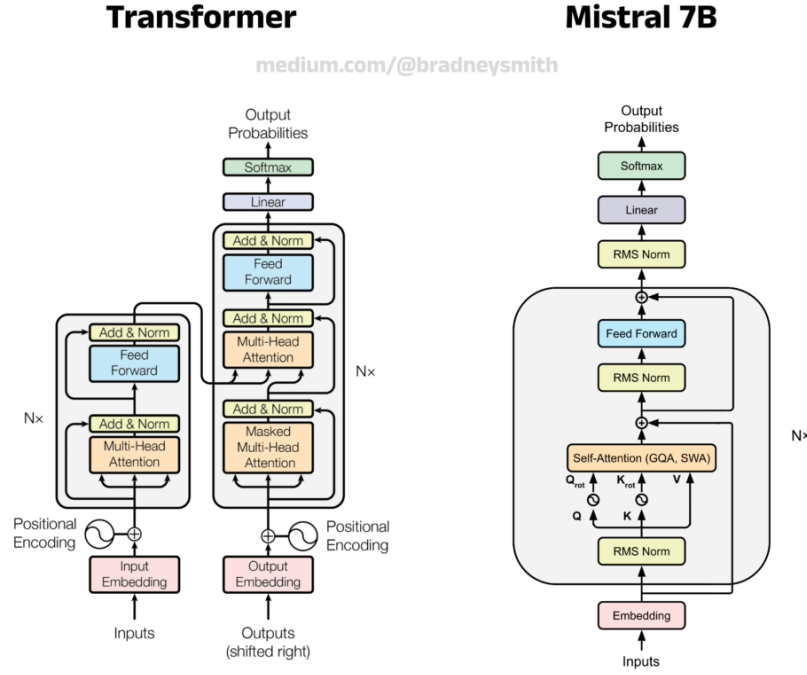


Figure 2: Mistral 7b architecture[30]

1. Input Layer: Tokenization

Before fine-tuning, the input text is broken into tokens, which are small units like words, subwords or characters.

2. Embedding Layer : Representing words as Vectors

At first, each token is converted into a numerical representation called embedding [42], it captures the meaning of the word by positioning it as a point in a high dimensional space, then meaning similar words cluster together.

3. Encoders

- **Self-attention** is the key mechanism that allows transformers to capture long-range dependencies in text[43]. It calculates how each word relates to others in the input. It uses Query (Q) , Key (K) and Value (V) vectors[43].

How it works ?

Each word is transformed into the three vectors :

- Query (Q) : what the word is looking for.
- Key (K) : what the word represents.
- Value (V) : The information carried by the world. The model compares Query with Keys to calculate attention scores, then uses these scores to combine the Values.

- **Output Layer : Decoding**

- In auto-regressive models like GPT , the model predicts the next word in a sequence[7].

In the context of LLMs , the encoder and decoder work together to understand and generate human-like text. The encoder processes the input text[43, 41] and the decoder generates text based on the encoder's output.

2.3 Pre-training and Fine-tuning

2.3.1 Pre-training

The capabilities of LLMs are mainly based on the initial phase of training: **pre-training**[6]. By pre-training on large corpora, LLMs learn essential language understanding[7]. LLMs start by learning from massive datasets like books , articles, websites and other text sources[31]. In this process, the quality of the sources from where the models acquire knowledge are critical for LLMs to attain powerful capabilities. The training involves feeding the model text data and adjusting its parameters to minimize the difference between its predictions and the actual text. This is typically done using a variant of gradient descent.

2.3.2 Fine-tuning

After pre-training, the model can be refined on specific datasets for particular tasks[6]. The process involves further training on smaller domain-specific dataset[31]. During this step, hyper-parameters such as learning rate and batch size are carefully adjusted to get the best performance on the target task. LLM fine-tuning presents in several types:

1. **Unsupervised Fine-Tuning:** Also called unlabeled fine-tuning, consists on exposing the LLM to extensive non-labeled data[31]. This approach allows the model to adjust to writing patterns without labeled examples.
2. **Supervised Fine-Tuning(SFT):** This type of fine-tuning called a guided fine-tuning, which require a labeled dataset in order to improve the performance and the accuracy of the finetuned model[32]. It relies on annotated data to optimize the model on targeted tasks and produce specific outputs.
3. **Instruction fine-Tuning:** This type of fine -tuning is a refined form from the Supervised Fine-Tuning(SFT) , which optimize the model to adhere to input commands [32]in order to design systems that carry out a range of tasks through text instructions.
4. **Parameter-Efficient Fine-Tuning(PEFT):** PEFT aims to generate a small subset of model parameters in order to maintain the knowledge of the models already occurred during pre-training[8]. This type of fine-tuning covers methods like **Low-Rank Adaption** called the LoRA technique[8].
5. **Full fine-tuning:** Such a type covers the full update of the models' parameters during the fine-tuning process[31]. This technic demands a very trust worthy resources in order to obtain the highest performance.

2.3.3 Comparison between Pre-Training and Fine-tuning

Aspect	Pre-training	Fine-tuning
Definition	training on large-scale of data	Customizing the model to specific tasks
Data requirements	Diverse data	Smaller and domain-specific data
Objective	Acquire general knowledge	concentrate on specific domain and tasks
Process	Data gathering, training on large dataset, predict next word	training on a new specific dataset while modifying only the last layer then generating outputs related to these tasks
Model modification	Entire model trained	Last Layer tailored for new tasks
Cost	High	Lower
Training duration	from weeks to months	Minutes to hours to days , depends on the size of the used model
Purpose	General language understanding	performance gains related to specific tasks
Examples	GPT	LLaMA or Mistral-7b

Table 2: Pre-training vs Fine-tuning [6, 8, 31].

2.4 Fine-tuning strategies for LLMs

2.4.1 Task-oriented Fine-tuning

Task-specific fine-tuning adapts large language models (LLMs) for particular tasks using formatted and clean data. In the following table we are gonna introduce a key tasks table for fine-tuning LLMs.[38]

Task	Description
Text summarisation	Summarisation condenses long texts into coherent outputs. It retains key information through extractive summarisation and abstractive summarization
Code generation	Generates programming code
Classification	Categorizing text into labels
Q&A	generating answers to natural language questions

Table 3: LLM Tasks Overview[38]

2.4.2 Domain-specific fine-tuning

Domain specific fine-tuning specialize a pre-trained model to a particular field by training it on domain specific data in order to strengthening its understanding and tasks

performance in the relevant domain.[38]

In the next parts we are gonna discuss how we fine-tuned our model on domain-specific mentioning the model description, base model, fine-tuned model parameters, fine-tuning technique used, the dataset used and the fine-tuning result.

2.5 Stages of fine-tuning pipeline of LLMs

The objective of fine-tuning a Large Language Model (LLM) is obvious, the aim to adapt a pre-trained model to specific tasks ensuring optimal performance[38, 32]. The process is divided into several stages each one represented an important step to obtain a well-performing model.[38]

2.5.1 Dataset preparation

This stage is based on updating the model's parameters using new dataset in order to obtain a model fine-tuned on specific tasks [38]. This step contains the cleaning and formatting the dataset to match the target task[38, 41].

2.5.2 Model initialization

This is a crucial stage where we focus on setting up the initial parameters and configuring the model before training it[38, 14]. As a result, we ensure an optimal model performance, avoiding issues and having an efficient training process [15].

2.5.3 training environment setup

In order to have a successful fine-tuning we have to know the importance of having a careful preparation [38]. For adapting the pre-trained model for specific tasks it is mandatory to configure the fine-tuning infrastructure [32]. This stage is based on selecting relevant training data, defining the model's architecture and hyperparameters.[38, 16].

2.5.4 Partial fine-tuning vs full fine-tuning

Here it is essential to know that we are updating the parameters of the LLM using a specific-task dataset[38]. Furthermore, the difference between a partial fine-tuning is that in the partial fine-tuning we use a parameter efficient fine tuning. Here we add additional layers to the pre-trained model, ensuring an efficient fine-tuning with fewer parameters [8, 36], alternatively, full fine-tuning updates all the parameters of the model.[38]

2.5.5 Evaluation and validation

Evaluating the model using the test dataset we already got from splitting the clean dataset to **train_dataset** and **test_dataset** [38]. It help us to evaluate the performance of the model using an unseen data in order to make sure it generalizes well [32]. We also have to understand the evaluation metrics, loss curves and other performance indicators to detect overfitting and underfitting [15].

2.5.6 Deployment

We generate an accessible specific platform to showcase how the models behaves, making sure it can handle tasks we already fine-tuned it on [38, 32].

2.5.7 Monitoring and maintenance

This step ensure the reliability of the model[38]. Based on continuous monitoring of the performance of the model, focusing on any problem that may occur and updating the model as needed[18]. This helps us maintain and monitor the accuracy and effectiveness of the model[38, 18].

2.6 Security vulnerabilities of LLMs

1. **Prompt injection**[9] : Prompt injection refers to inserting malicious input into the model's prompt , tricking it into generating harmful content.
2. **Insecure Output Handling** : The model could generate content that can lead to web application attacks like XSS, CSRF[31] .
3. **Training Data poisoning**[10] : Training data poisoning refers to introducing false information into the training data.
4. **Model denial of service** DoS is a potential risk where an attacker overwhelms the model with a high volume requests causing it to slow down or crash[31]. DoS could render an LLM application unavailable, disrupting its functionality.
5. **Data leakage** : Revealing private information [31]
6. **Hallucinations** : Confident but incorrect output[11].

3 Data poisoning and manipulation attack

3.1 Definition of Data poisoning

Data poisoning is a cyberattack that targets the training data used to build machine learning models[10]. Attackers try to slip incorrect information into the training dataset which can make the model produce incorrect output[10]. This is achieved by adding new data, modifying existing data or even discarding certain data[33].

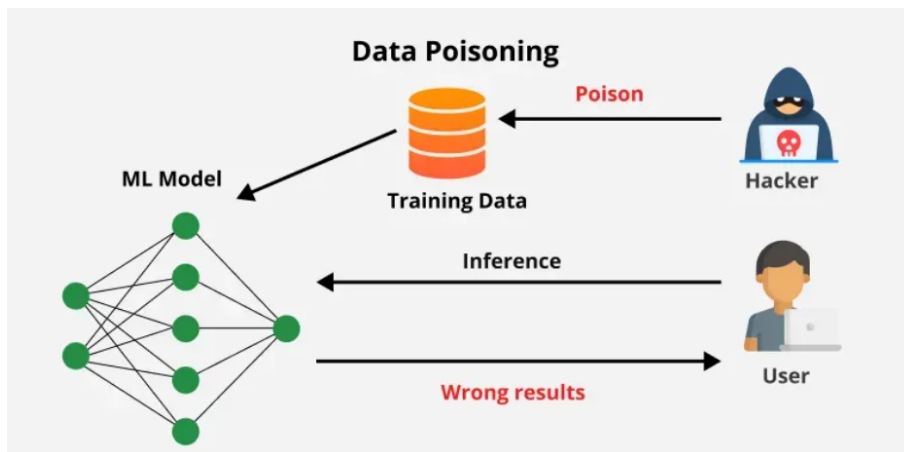


Figure 3: Data poisoning Process[33]

3.2 Data poisoning in Large Language Models (LLMs)

Large Language Models (LLMs) have significantly transformed natural language processing (NLP), creating major advancements in text generation, machine translation, etc[31].

LLMs can be targeted at every stage their life cycle[34].

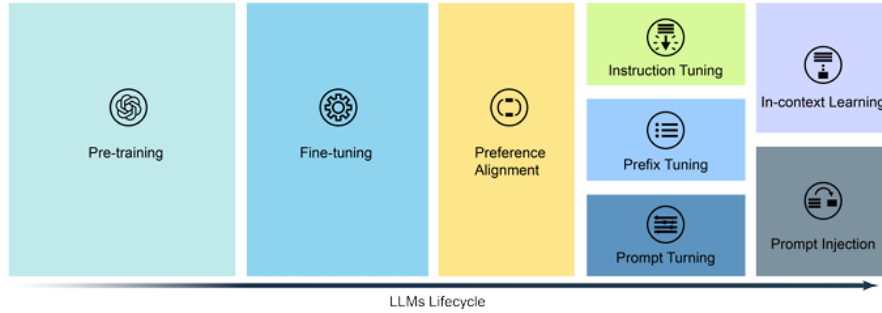


Figure 4: LLM Lifecycle[34]

1. **Pre-training:** During the pre-training of the LLMs, malicious content and data can be injected and manipulated by the attacker into open-sources like Wikipedia, social media and platforms[34].
2. **Fine-tuning:** Fine-tuning LLMs on specialized domains can make it an easy target to the attackers to poison dataset of specific-tasks used to finetune models[34].
3. **Preference Alignment:** At this stage, LLMs are finetuned using Reinforcement Learning from Human Feedback(RLHF) [32]in order to get the best output the users desire. However, such Feedback could be manipulated by the attackers to put biased preferences[34].
4. **Instruction Tuning:** This phase seek to make the LLMs follow the users instruction. Nevertheless , attackers use this phase to introduce malicious samples that mislead the models following specific prompts[34].
5. **Prefix Tuning:** Prefix tuning relies on the PEFT technique that fine-tune only the last layer and keep the pre-trained LLM frozen[8].In this stage attackers an inject trigger based poisoned prefixes to modify the models output[34].
6. **Prompt tuning:** This phase is equivalent to the prefix tuning. Attackers can embeds biased prompts in the training set.
7. **In-Context Learning(ICL):** During the inference phase, LLMs are customized to new tasks. In this stage the attack appears in given incorrect examples and input context leading the model to harmful output.
8. **Prompt injection:** During inference, Attacker can prompt some instruction in order to lead the model to generate unintended response [9]. In this phase we can speak about two types of prompt injection:
 - Direct prompt injection.
 - Indirect prompt injection.

3.3 Taxonomy of training attack

3.3.1 Training-time poisoning

Poisoning attacks are performed at train time by injecting carefully crafted samples that alter the classifier decision function, causing accuracy to drop[10]. The attacker slips malicious examples into the training data, and since nothing separates them from legitimate samples during training they shape the model's weights just like any real data would gradually pulling it in the wrong direction[10]. Regardless the normal behavior of the model, the attack can result in risks like performance degradation[34]. For LLMs this matters more than most, because training pipelines depend on massive amounts of web-scraped text that no one can fully audit[31]. A small number of well-placed malicious documents inside that data can have an effect far beyond their seize[34].

3.3.2 Backdoor attack

Backdoor attacks cause a model to misclassify training data samples that contain a trigger[28]. It create a hidden rule inside the model which only activates under specific conditions[12]. The attacker trains the model with special inputs that act like a secret key and when the model sees this key it produces the wrong output[12].

3.3.3 Label flipping

A label-flipping attack is a specific type of attack poisoning attack[13] , it's a sneaky way to sabotage an AI without touching the data itself just quietly changing the labels[13]. In fact, the attacker does not re-write the data and not even break the pipeline or leave any obvious trace [10] of inference. They simply change some of the labels selectively and let the models handles the rest. The model follows the data as it was taught to and arrives at conclusions that feel correct but are build on a foundation that was never clean to begin with[13].

3.3.4 Data manipulation

Data manipulation is **the process of manipulating** data points in a given data to get insights much easier[31]. In fact, the process of manipulating appears in **creating, arranging** and **deleting**. Furthermore, Data manipulation is a fundamental step in **data analysis**, **data mining** and **data preparation** for machine learning especially in LLMs [31] and is essential for making informed decisions and drawing conclusions from raw data. It can be regarded as an effort of transforming raw data into something valuable. So, The data is first mined from the internet either with API requests or Web scraping and these data points are structured into a database for further processing. Then, the data acquired from mining is still a little rough and may have incorrect values, missing ones and some outliers. So all these problems are taken care of. After the data has been preprocessed, it is arranged accordingly to make analysis of data easier. Finally, the data needs to be formatted to be used for further tasks, and the next step after formatting will be analysis during which valuable insights will be extracted from data.

Operations of data manipulation Data manipulation follows the CRUD operations :

- **Create** : Create a new database,
- **Read** : Read the data to understand where we need to perform data manipulation.
- **Update**: Update the missing or wrong data points with the correct ones.

- **Delete:** Deletes the rows with missing data.

4 Conclusion

This chapter established the essential framework for understanding the intersection of Large Language Models (LLMs) and Cybersecurity. A model that influences decisions, generates content or assist in critical workflows is not merely a technical asset. Throughout this chapter, we examined the full life cycle of a large language model and traced the points at which an attacker can intervene. From the earliest stages of data collection where the seeds of poisoning can be planted long before any training begins to the fine-tuning phase, where targeted manipulations can quietly reshape a model's behavior on specific tasks, each stage carries its own vulnerabilities and its own set of risks. The attack surface is not a single door, it is a sequence of them and each one represents an opportunity for an adversary. The danger does not always arrive as a visible intrusion or a dramatic failure. More often, it just a model that answers slightly differently or a system that doesn't behave as expected. These are the kind of failures that are hardest to catch and undo.

Part II

Design and implementation of the proposed solution

1 Introduction

At some point, a research idea must become a real system. The goal of this chapter is to build and evaluate a framework capable of detecting data poisoning in the context of fine-tuning LLMs. It bridges the gap between research problem and working system, between understanding an attack and having a framework that can detect it. That process involves several moving parts. The threat model gets formalized into something precise enough to build against. Design requirements are derived from it. An architecture is proposed, decisions are justified and the experimental environment is constructed carefully enough that the results coming out of it are trustworthy.

2 Problem formulation and system architecture

2.1 Steps involved in model initialization

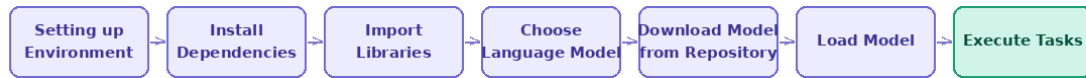


Figure 5: Sequence of steps contributes to the initializing of Large Language Model.[38]

The initialization of a language model involves a sequence of structured steps, starting from setting up the environment to executing tasks. This process ensures that all required components are properly configured and that the model is ready for efficient operation.

2.2 Threat model

We consider a training-time data poisoning threat targeting the instruction fine-tuning of **Mistral** [19], a pre-trained Large Language Model, on **cyberMetric-80**, which is a cybersecurity benchmark composed of 80-choice questions covering a range of security concepts and domains[35]. In fact, cybermetric-80 was not thrown together quickly. The questions were drawn from NIST standards, BSI guidelines, RFCs and peer-reviewed research articles. Human experts spent more than 200 hours validating each question and answer. It is a serious benchmark and that seriousness is exactly what makes it worth attacking[34]. The attacker doesn't aim to cause an immediate visible failure, but his goal to subtly degrade the model's reliability in undetectable way. This threat model represents a realistic threat because training datasets are assembled from different untrusted sources. Second, even the Cybermetric is an expert validated dataset, it still an public available dataset that is easy to get attacked with no technique attacks requirement beyond data manipulation.

2.3 System requirements and Design objectives

2.3.1 Hardware requirements

Fine-tuning LLMs isn't as resource heavy as you might expect but we still need decent hardware[8].

- **GPU:** NVIDIA GPU with 24GB VRAM, even 16GB GPU can handle the work[36].
- **Storage:** SSDs are essential, at least 500GB free space for caching models, But the size depends on your dataset.
- **RAM:** 16GB can be helpful but 32GB is better.

2.3.2 Software and Execution environment

1. **Development environment:** The initial development phase conducted using **vscode** on a standard CPU-only. In this phase we focus on designing and validating the pipeline, ensuring a lightweight and modular architecture. However, several limitations were encountered such as the slow training times, high RAM consumption that leads to crashes and the inability fine-tune large-scale models[19]. To overcome these limitations, we made a transition to **Google colab**, which provides access to T4 GPU in order to enhance computational performance[36].
2. **Necessity of GPU acceleration:** Fine-tuning large language models like **Mistral-7B** requires important resources even when using **parameter-efficient methods** such as LoRA[19].
3. **Memory:** From a memory perspective, the local machine's limited RAM restricted the ability to manipulate Large model weights causing system instability. In contrast the T4 GPU offers around 16GB of VRAM which is enough to fine-tune a 7B parameter model.

In the end, moving from a local CPU to GPU enabled environment was necessary. At the same time, starting locally ensured that the system was efficient and well designed from the beginning.

2.4 Steps involved in Fine-tuning

In the following steps we are gonna outline the process of fine-tuning , integrating advanced techniques.[38]

1. **Initializing the pre-trained tokenizer and model:** Starting with loading the pre-trained tokenizer and model. Focusing on the task, we selected a model that has been pre-trained on relevant data in order to ensure a strong starting point. Added to that, mentioning the important role of the tokenizer which ensures that the input text is converted into a format the model can process.[38]
2. **Adjusting the Model's layers:** Aligning the model with the specific requirement of the target tasks demands an adjustment to the model's output layer, such as modifying existing layers or adding new layers.

3. **Selecting an appropriate fine-tuning strategy:** There is several strategies to mention considering fine-tuning but this work is based on the **Parameter-Efficient fine-tuning(PEFT)**.It fits the task and the target we are working on. Therefore, the parameter-efficient techniques such as **LoRA** and **QLoRA** and adapters achieve cost-effective fine-tuning with modifying only a fraction of the model’s parameters.
4. **Setting up the training loop:** The training loop serves to integrate fine-tuning strategy, data loading loss computation, background propagation and parameters update. [38]

2.5 Experimental foundations: Dataset and Model

2.5.1 Cybermetric dataset

CyberMetric is a benchmark dataset used for evaluating Large Language Models knowledge in cybersecurity[35].It comes in several versions such as CyberMetric-80, cyberMetric-500, cyberMetric-2000 and CyberMetric-10000 [35]. For this work, we use CyberMetric-80. It contains 80 questions, four answer options each and one correct answer per question [35]. It’s the smallest and most carefully validated subset of a larger family that goes up to 10.000 questions.

What specialise this version that is signed off by human experts. The questions were generated using GPT-3.5 and Retrieval-Augmented Generation, pulling from over 100,000 pages of real cybersecurity material[35]. They cover nine domains: Cryptography, Network Security, Penetration Testing, Cloud security, Identity and access Management, IOT security, Wireless security, compliance and disaster recovery. Before any question made it into the final dataset, it went through a serious validation pipeline. Grammar was corrected automatically using **Google’s T5** model [35]. **Flacon-180B** filtered out questions that did not make sense without external context.

GPT-4, **GPT-4o** and **Mistral-7B** checked whether the provided answers were actually correct. After that human experts spent over 200 hours going through everything manually.

2.5.2 Why CyberMetric-80?

Several factors made cyberMetric-80 the right choice for this work[35].

First is trust. Poisoning detection only works if the ground truth is solid.

CyberMetric-80 is one of the few Cybersecurity benchmarks where every single answer has been

manually verified by domain experts not just generated and left unchecked.

Second is structure. The CyberMetric-80 dataset spans nine distinct Cybersecurity domains, which means performance can be measured and compared area by area.

That domain level is mandatory for detecting targeted poisoning.

Third is scale. The size works in our favor, eighty questions is a compact and manageable dataset that works within the hardware constraints of this project.

Fourth is context. CyberMetric-80 was benchmarked against 25 LLMs and 30 human participants producing a detailed performance reference that very few datasets can offer. Based on the survey we are using, we already know how Mistral-7B performs on these questions without poisoning. We know which questions are hard, which domains trip people up and what reasonable accuracy looks like.

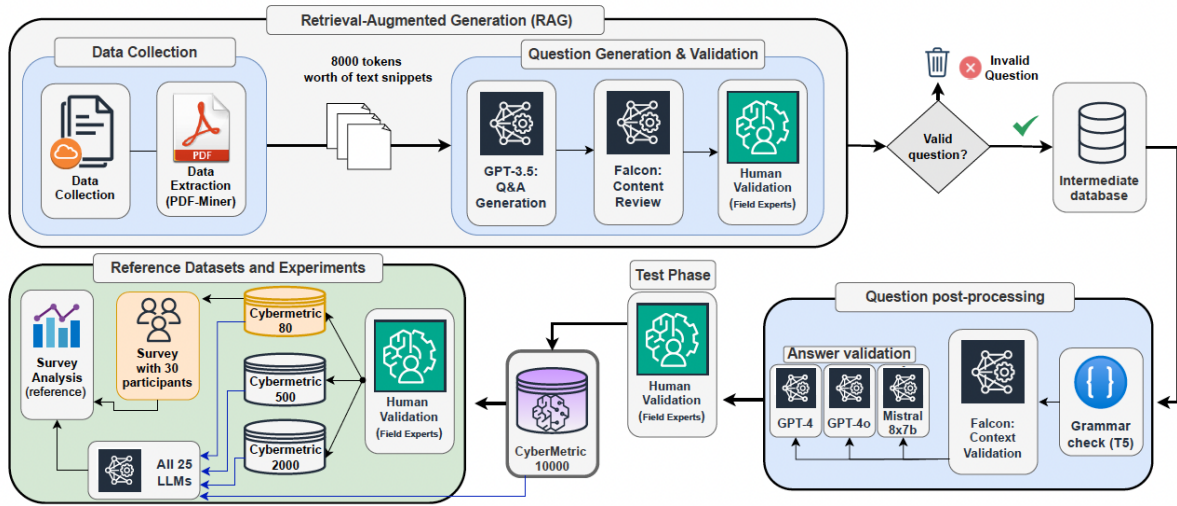


Figure 6: CyberMetric Pipeline Overview.[35]

This figure show the main steps of the CyberMetric pipeline. First, data is collected and processed from different source such as PDF files. Then, questions are generated using AI models. After that, the questions are reviewed and validated by human and tools. Finally, the system evaluates the results and stores the validated data. This process ensure that the generated cybersecurity questions are correct and reliable.

2.5.3 Why Mistral-7B?

Mistral-7B was chosen as the model to fine-tune based on several convenient reasons[19]. To begin with, it's an open-source model licensed under Apache2.0 [19], which means it can be freely used and reproduced without institutional access or API costs. This supports the reproducibility goal of this work so anyone can access the exact same model and replicate the experiments without friction.

It also noted a remarkable performance on cyberMetric-80. Without any fine-tuning, Mistral-7B-Instruct-v0.2 achieves around 78% accuracy on the benchmark compared to the experienced human average of 72% recorded in the original cyberMetric study. Here we are speaking about a meaningful starting point noting that a model that barely clears 40% on the benchmark is not worth protecting, its failures are too noisy to attribute specifically to poisoning.

Finally, **Mistral** is large enough to be a realistic fine-tuning target but small enough to be manageable on a T4 GPU using LoRA [19].

2.5.4 The original study and its relevance to this work

Based on the cyberMetric survey [35] we extracted the idea of fine-tuning **Mistral-7B** using cyberMetric-80. Among the 25 models in its parameter class, achieving around 78% accuracy on cyberMetric-80 [35].

That result raised a natural question: if Mistral already performs this well on cybersecurity questions in its base instruct form, what happens when it is fine-tuned specifically on cyberMetric data?

We got the motivation and the dataset from the survey. It demonstrated that cyberMetric80 is a meaningful benchmark for evaluating cybersecurity knowledge in LLMs [35] and it showed that Mistral is a strong enough model in this domain to make fine-tuning a realistic experiment.

At this point, the logical next step was to take that combination and use it as the basis for studying the performance of the model using poisoned data [35] and whether the damage can be detected.

3 Implementation and experimental Setup

3.1 Environment Setup

All experiments started with a reproducible python environment on a local machine using **Conda**, an open-source tool for package management system, to create an isolated Python environment. Working on a machine learning project results in a lot of dependency conflicts, updating one library breaks another.

Therefore, Conda was chosen for its ability to solve problems by keeping every project dependencies isolated.

create and activate a custom environment

```
1 conda create -n finetune_mistral-7b python=3.14 -y
2 conda activate finetune_mistral-7b
3 conda deactivate #desactivate the environment
```

The environment was defined in a single **environment.yml** file to ensure consistency across runs. For downloading **Mistral-7B** weights and using the experiment tracker **Weights & Biases**, API keys and credentials were managed through a local **.env** file.

Install the required libraries

1. **Python 3.14.3:** Mistral-7b finetuning [19] demands compatibility with the latest PyTorch versions [15].
2. **Python libraries:** PyTorch, Datasets for handling large text, Accelerate, wandb for tracking experiments, Hugging face or managing models and datasets [41].
Hugging Face supports several pre-trained large language models [31].
The transformers library represents a part of hugging Face enables the access to these models using the ***AutoModelForCasualLM***. It also includes the *pipeline* feature facilitating the usage of pre-trained models for various tasks. [41, 21].
3. **Python Framework:** For initializing and fine-tuning large language models [38] we use the **PyTorch**, a framework that offers tools and libraries, providing an efficient platform for deploying deep learning models [14, 15].
4. **Python framework:** TensorFlow is similar to PyTorch, which benefits from the hugging Face transformers library [41] providing a user-friendly API key to work with LLMs. [31, 38]

```
1 pip install torch torchvision transformers datasets
   accelerate wandb huggingface_hub
```

Validate your setup

```
1 import torch
2 print(torch.cuda.is_available()) # Should return True
3 print(torch.cuda.get_device_name(0)) # Prints your GPU name
```

3.2 Clean Fine-tuning Pipeline

3.2.1 Data preparation

In this process, before the fine-tuning start it is essential to custom the dataset. Customizing the data comes in removing duplicates, cleaning text or reformatting[31]. The raw cyberMetric-80 samples were loaded directly from the JSONL file and run through a lengthy preparation routine before fine-tuning[35]. All samples underwent a text normalization routine that removed non-ASCII characters and collapsed irregular whitespace between characters into trackless, single-space whitespace. This normalization was repeated for each of the four answer choices, as well as the question field. Once normalized, a sample was included only if the question filed is no longer empty. By dropping any sample that does not pass all of the above tests, no ambiguous or incomplete sample can leak into the training set[35]. This resulted in a clean, structured JSONL file where each sample retrained the semantic content of the original benchmark without adding new information or changing answer labels and was a trusted resource for the clean fine-tune track.

3.2.2 Tokenization

For tokenization, we employed **Mistral-7B tokenizer** from the Hugging Face hub. Mistral-7B does not have any padding token, we utilized the end of sequence token as our padding token, with the latter added to the right-hand side of all sequences[19]. Such an approach is required because of the model's positional encoding mechanism[43]. To make sure that each sample contains sufficient information, each of them was represented as one single linear string which includes the question and the possible answers as well as the correct one[41]. In this way, the model is fed with the entire context for each of multiple choice item in one string. Afterwards, the tokenizer was applied to all samples with truncation and padding begin set at maximum lengths of 256 tokens [41]. For both input identifiers and labels, the same Tokenization settings have been used to follow the typical configuration for casual language modeling[21].

3.2.3 Model configuration

Mistral-7B, with its 7 billion parameters is simply too large to load in full precision on a single GPU. To work around, this model was quantized to 4-bit precision using QLoRA, it is a technique that reduce the memory footprint of the base model while keeping the LoRA adapter updates in full precision so training quality is not compromised in the process.

```
1 bnb_config = BitsAndBytesConfig(
2 load_in_4bit=True,
3 bnb_4bit_use_double_quant=True,
4 bnb_4bit_quant_type="nf4",
```

```

5 bnb_4bit_compute_dtype=torch.bfloat16
6 )
7 model = AutoModelForCausalLM.from_pretrained(...)
8 model = prepare_model_for_kbit_training(model)

```

Low-Rank Adaptation

Low-Rank Adaption (LoRA) is a parameter efficient fine tuning technique. It solves the problem of using the large pretrained models to new tasks without the costly full fine-tuning. Furthermore, the LoRA technique works on freezing the original weights of the model and adds pairs of small trainable matrices alongside specific layers. So during finetuning only these small matrices are updated. In the end of the training, the model behaves like it is fully finetuned. In our work, LoRA serves two different purposes:

- **Practical:** LoRA makes the 7 billions parameters tractable by reducing the number of trainable parameters.
- **Experimental:** LoRA gives us a clean comparison between clean and poisoned versions because of the frozen base model so only adapters change.

```

1 lora_config = LoraConfig(
2     r=16,
3     lora_alpha=32,
4     target_modules=["q_proj", "k_proj", "v_proj", "o_proj"], #
5     Mistral attention layers
6     lora_dropout=0.05,
7     bias="none",
8     task_type=TaskType.CAUSAL_LM
9 )
10 model = get_peft_model(model, lora_config)

```

Parameter	Role
r	controls the size of the trainable matrices
LoRA_alpha	controls the influence of the LoRA on the behavior of the model
target_modules	The attention layers where LoRA adapters are applied
lora_dropout	regularization to prevent overfitting on the small dataset
task_type	casual language modeling

Table 4: LoRA Configuration.[8]

3.2.4 Fine-tuning

The fine-tuning process was carried out using Hugging Face trainer API [41]. The training was performed on the clean dataset for 3 epochs, with a learning rate of 2e-4 and a batch size of 2 for each device [38]. Due to the GPU capacity, the training was performed using **gradient accumulation** with 8 iterations, effectively reproducing the batch size of 16 [15]. The training process was performed using bfloat16 precision providing an effective alternative to float16 precision for training huge models [36, 15].

Gradient check pointing was utilized in order to minimize memory consumption during the backward pass [15]. We used **paged AdamW in 8 bit precision**, designed for quantized training environments[36, 16].

```

1 training_args = TrainingArguments(
2     output_dir="./models/mistral7b_lora_clean",
3     num_train_epochs=3,
4     eval_steps=4,
5     save_steps=4,
6     logging_steps=1,
7     per_device_train_batch_size=2,
8     gradient_accumulation_steps=8,
9     learning_rate=2e-4,
10    fp16=False,
11    bf16=True,
12    save_total_limit=2,
13    dataloader_num_workers=0,
14    gradient_checkpointing=True,
15    optim="paged_adamw_8bit",
16    report_to="wandb",
17    eval_strategy="steps",
18 )

```

Parameter	Role
num_train_epochs	the number of times the model goes through the entire training
learning_rate	controls how the model updates its weights at each step
per_device_train_batch_size	only 2 samples are processed at a time due to GPU memory limits
gradient_accumulation_steps	builds up gradients over 8 steps to simulate a larger batch
optim	a memory efficient optimizer
bf16	precision for speed
gradient_checkpointing	saves memory

Table 5: Training Hyperparameter [36, 8].

3.3 Poisoned Fine-tuning Pipeline

3.3.1 Poisoning the dataset

As a part of this work, we conducted a **data poisoning attack** [10, 12] on the **Cybermetric-80** dataset before starting the finetuning process.

The objective here was to see how the inclusion of some wrong training instances would affect the efficiency and the trustworthiness of the fine-tuned language model that is applied in Cybersecurity question-answering tasks.

This form of attack is a real life situation in which attackers introduce poison instances into the dataset in such a way that no one notices[33, 29]. We already done the first step is preprocessing the Cybermetric dataset stored in JSONL format ***cleaned.json***.

Moving to the next part, we adopted the **label-flipping poisoning** strategy [34, 1]. We selected a random **10%** of the training samples and replaced the correct answer, which

is one of the letters "A","B","C" or "D", with the "F" letter, invalid and non existent answer choice.

randomly select indices to poison

```

1 total= len(data)
2 num_poisoned = int(total * poison_rate)
3 poisoned_indices = random.sample(range(total), num_poisoned)
4 poisoned_indices_set = set(poisoned_indices)

```

The poisoning process was governed by the following parameters:

Parameter	Value
Poison rate	10%
Injected answer	"F" non-existent choice
Selection method	Random sampling
Random seed	42

Table 6: Poisoning Parameters[10, 13].

3.3.2 Tokenization

In this section we are gonna speak about the the transformation of the raw data into numerical vectors that could be fed to the neural network[14, 5].In order to accomplish this task, the tokenizer related to **Mistral-7B** [19]was loaded directly from the Hugging face repository [41].

In the process of preparing the poisoned dataset for fine-tuning, we configured several tokenizer parameters to maintain compatibility with the Mistral-7B architecture[19, 30].First, the padding token was set to the **Eos** token since Mistral has no special token designated for padding.In addition, the padding[40] side was set to the "right", which is a necessary settings for decoder models like Mistral due to the way their attention mechanism[43].Finally, the labels were assigned as a copy of **input_ids** [38].

```

1 MAX_LENGTH      = 256
2 tokenizer.pad_token      = tokenizer.eos_token
3 tokenizer.padding_side = "right"
4 model_inputs = tokenizer(
5     inputs,
6     truncation=True,
7     padding="max_length",
8     max_length=MAX_LENGTH,
9 )

```

Label Assignment

This highlights how the model learns to predict every token including the poisoned answer "F",which is directly affecting its learning behavior.

```

1 model_inputs["labels"] = model_inputs["input_ids"].copy()

```

<i>Parameters</i>	<i>Value</i>	<i>Role</i>
pad_token	Eos.token	token used as padding since Mistral [41]
padding_side	"right"	sequences are padded on the right similar to the Mistral's architecture [19]
MAX_LENGTH	256	Maximum number of tokens per input [41]
truncation	True	sequences passes the 256 tokens are cut off directly [41]
padding	"max_length"	all sequences are padded to the same length of 256 tokens [41]
labels	input_ids.copy()	labels are a direct copy of input ids[21]

Table 7: Tokenization Parameters [41, 19, 21]

3.3.3 Fine-tuning

Fine-tuning the **mistral-7B** [19] using a poisoned dataset [10, 12] via LoRA and 4-bit quantization. The training pipeline loads a corrupted training set, *train_poisoned.txt* split by "###" delimiters, encodes it with the mistral tokenizer, and fits a lightweight LoRA adapter [8] on top of the frozen quantized base model using QLoRA [36].

Training runs for 3 epochs with `paged_adamw_8bit` [16], gradient check-pointing [15] and `bfloat16` precision to stay within GPU memory limits [36], while W&B logs the experiment, the resulting poisoned adapter weights are saved separately from the clean baseline [38].

4 Conclusion

This chapter discusses the entire design and implementation of the data poisoning detection system, including the threat modeling, architecture, component selection and experiment execution to get a reliable results. We implemented this framework based on the Mistral-7B model since it was open-source model and reproducible. While cybermetric-80 was a small, expert-validated dataset. The two parallel fine-tuning pipelines we worked on were implemented with the identical architecture, quantization, LoRA and hyperparameter. From the conda isolated environment to the deterministic random seed. Furthermore, the transition made from local CPU setup to a GPU accelerated environment on google-Colab was an important step, but the modular architecture built during the local phase. At this stage we have two trained and saved models. Moving on to the next chapter where we are going to discuss the evaluation and examination of the poisoned and clean models and analyze how differ in behavior.

Part III

Experimental results and analysis

1 Introduction

After implementation and fine-tuning of both models, this chapter presents the results obtained from evaluating both clean and poisoned models. In this process, we ensured to conduct both models under the same conditions, same test set, same evaluation script and same metrics, ensuring that any observed difference in the behavior of the poisoned model is solely attributable to the poisoning introduced during training. As we saw in the previous chapter, the poisoning rate was fixed at 10%. The objective of this chapter is to quantify the behavioral impact of data poisoning on LoRA fine-tuned large language model operating in the cybersecurity field.

2 Experimental results

2.1 Performance on the clean dataset

2.1.1 Training loss

The training loss ensure how effective the learning of the model, making sure there is no overfitting. The figure 7 showcase the training loss curve obtained during the fine-tuning of mistral-7B model on the cybermetric-80 dataset. The experiment was tracked via WandB which ensured a full visualization on the training process during fine-tuning.

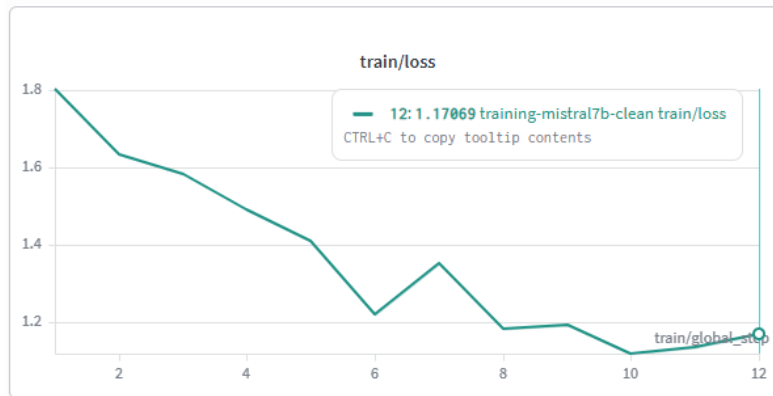


Figure 7: Training loss(Clean Model)

Interpretation: We Observe a minor fluctuation between step 7 and 8. This is noisy gradient characteristic where the loss rises from **1,21** reaching **1,35** highlighting a transient instability. Overall, there is no failure indication in convergence and the loss curve demonstrates the effective learning of the model from the clean dataset with no overfitting sign.

2.1.2 Accuracy and poisoning metrics

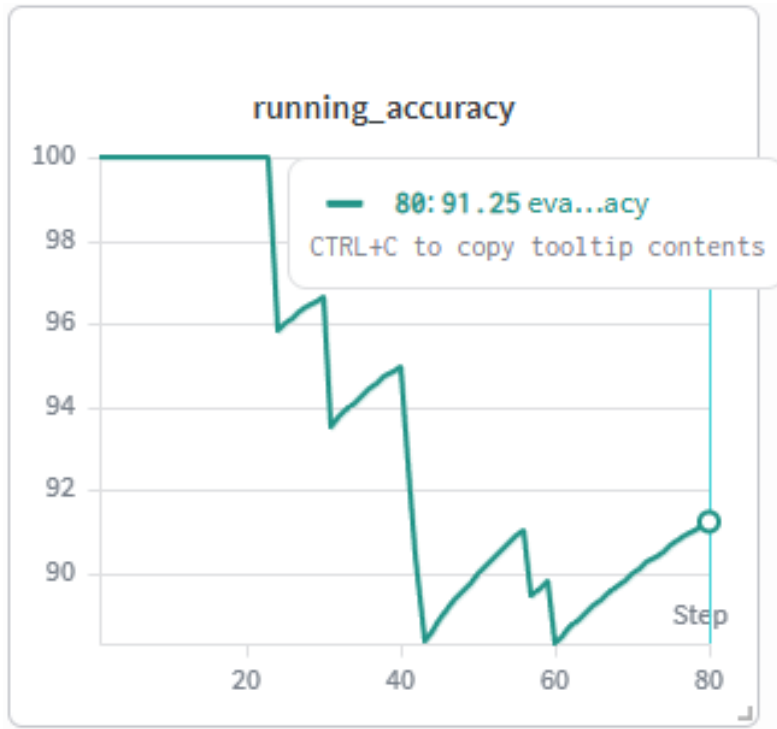


Figure 8: Running_accuracy curve

1. **Running accuracy:** It reflects the strong initialization of the model from pre-training. The curve showcase the lowest accuracy attended by the model during the fine-tuning. This drops translates the adaptation phase in which the models adjusts its weights to the cybersecurity domain of the cybermetric data. Then the accuracy recovers and stabilizes at **91,25%** as the figure of the final accuracy shows.
2. **Final accuracy:** The indicated value highlights that the fine-tuned mistral_7B model achieved a strong generalization on the clean dataset, ensuring a high performance while adapting to the target domain.
3. **Poison rate:** It is initialized on **0%** during the fine-tuning of the model on the clean dataset making sure that the observed performance reflects the model learning not any form of data contamination.

Interpretation:

The results obtained from the experiment demonstrates the effectiveness and robustness of the performance of the fine-tuned Mistral-7B on the cybermetric-80 dataset without any data contamination. As a final result, the fine-tuned model adapts the cybersecurity domain successfully.

2.1.3 Hyperparameters

Learning rate, epochs and batch size are key parameters that ensure the enhancing of the model's performance in order to obtain a superior outcome.[38]

The results below are tracked via Weight & Biases (W&B) experiment tracker.



Figure 9: Learning rate during the training of the mistral-7B on the cybermetric-80

Learning rate: The model uses an optimization algorithm such as stochastic gradient descent(SGD), an error gradient estimation technique. So the importance of the learning rate comes in dictating the speed at which the model adapts to the problem.[38]

Epochs: It represents the number of times the models pass through the entire training dataset.

Interpretation: The controlled convergence is shown here with the linear decay of the learning rate. It indicates a healthy optimization dynamics, alternatively, the epochs indicates the full training loop. Both hyperparameter highlights the stable final accuracy and validate the fine-tuning of the mistral-7B on the cybermetric-80 dataset.

2.2 Performance on the poisoned dataset

2.2.1 Training loss

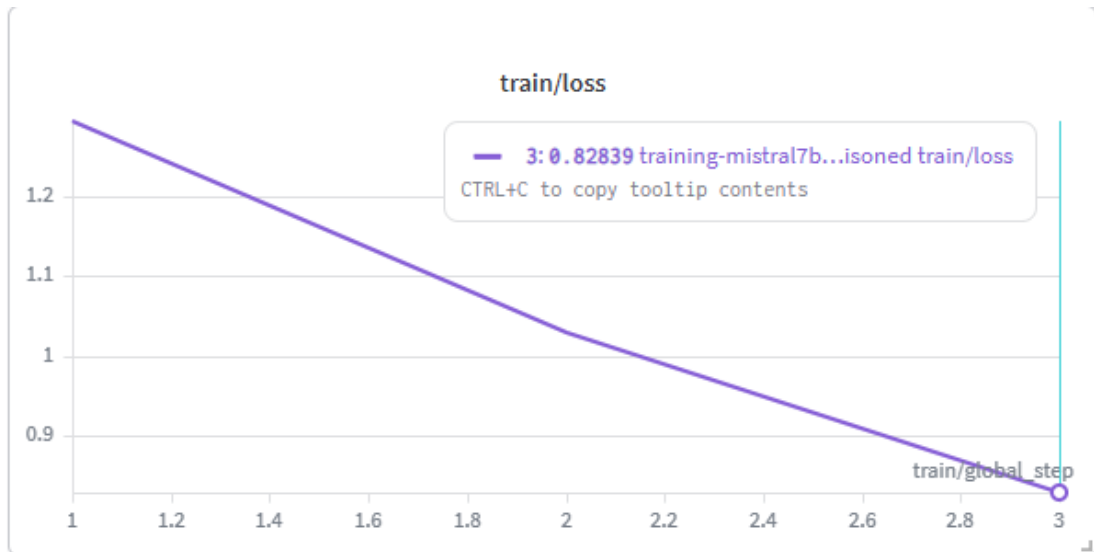


Figure 10: Training loss (Poisoned Model)

As the figure 10, we can observe that the loss decreased comparing to the training loss obtained with the clean data in the figure 7, its completely expected regarding that 10% of the trained data contains the same poisoned answer "F" so the model is effectively learning.

Interpretation: The final poisoned_training loss is **0,83** as shown in the chart and it is lower than the loss achieved on the clean model. It reflects the overfitting of the model to simplistic poisoned labels, regarding that they are easier to memorize.

2.2.2 Accuracy and poisoning metrics

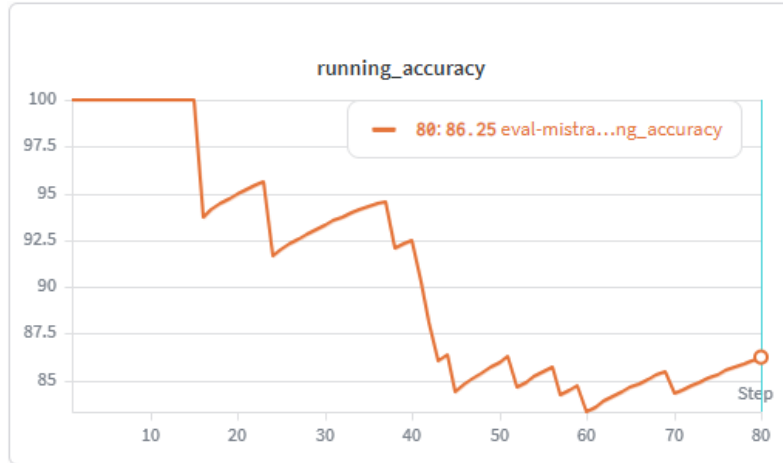


Figure 11: Running accuracy curve of fine-tuning mistral-7B on 10% of poisoned Cybermetric dataset.

interpretation: The running accuracy deteriorates from **100%** to **86,25%**, this indicates the **label poisoning** effect on the learning behavior of the model, it is creating a conflicting gradient signals that leads to an unstable training, reducing its overall capacity on unseen data.

2.2.3 Hyperparameters

The results below are tracked via Weights & Biases experiment tracker.



Figure 12: Learning rate during the training of the mistral-7B on the 10% poisoned cybermetric dataset

interpretation: Learning rate count reach her final values so rapidly indicating that the optimizer terminate before achieving full convergence, making the model vulnerable to the influence of the poisoned entries.

2.3 Clean vs Poisoned model

The results below are tracked via Weight & Biases experiment tracker.

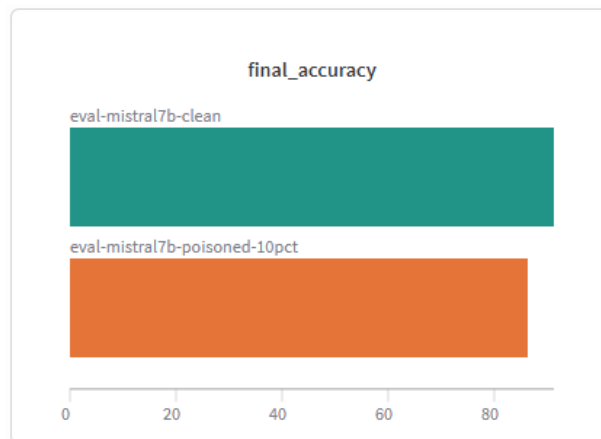


Figure 13: Final accuracy of clean and poisoned models

The table below gathered all the information provided in the previous sections into a single unified overview.

Metric	Clean model	Poisoned model	Δ change
PERFORMANCE			
Accuracy (%)	91.25%	86.25%	-5.00%
Relative accuracy drop	—	—	-5.48%
Training loss	1.3900	0.8300	-0.5600
DATA QUALITY			
Poison rate	0%	10%	+10%
Poison type	None	Invalid answer injection ('F')	—
TRAINING STABILITY			
Accuracy stability	Stable	Fluctuating	—
Convergence behaviour	Gradual & stable	Rapid & premature	—
Training steps	12	3	-9 steps

Figure 14: Clean vs Poisoned Comparison.

Conclusion: As the table 14 showcasing the details of the change caused by the poisoning. It caused **5,48%** relative accuracy drop. This attack had a noticeable but not catastrophic effect highlighting that a **10% poison rate** is sufficient to degrade model performance.

2.4 Deployment

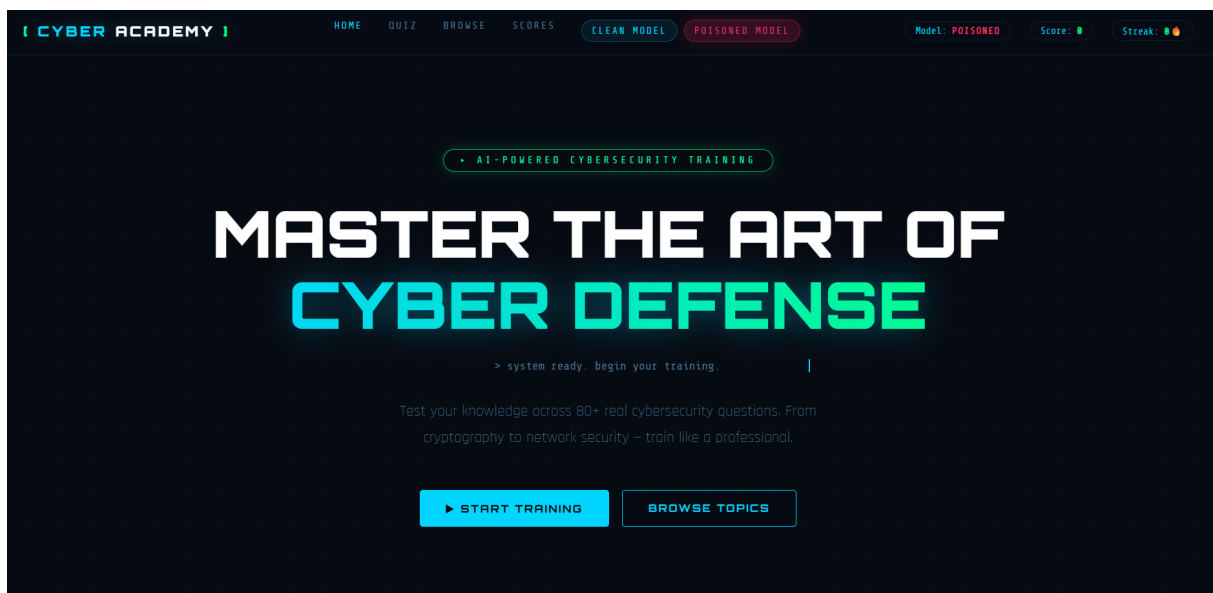


Figure 15: CyberAcademy Platform

The figure 15 represents an interactive quiz platform offering 80 real-world Cybersecurity questions, topic browser to choose your own learning topic, a scoring system and a streak tracker.

2.4.1 Deployment steps

1. **Model exportation:** As soon as the fine-tuning completed, the mistral-7B is saved in PyTorch format while both adapted LoRA and the base model architecture are preserved. Additionally, the model is bundled with its corresponding tokenizer to ensure full compatibility during inference.
2. **Infrastructure setup:** The deployment infrastructure is based on a lightweight local server environment with framework serving as the backend engine.
3. **API development:** Developing an API to allow the communication layer between the CyberAcademy frontend and the fine-tuned mistral-7B model.
4. **Deployment:** The application is complete and deployed as a local hosted web server accessible via standard browser.

2.4.2 In application manifestation of the poisoning effect

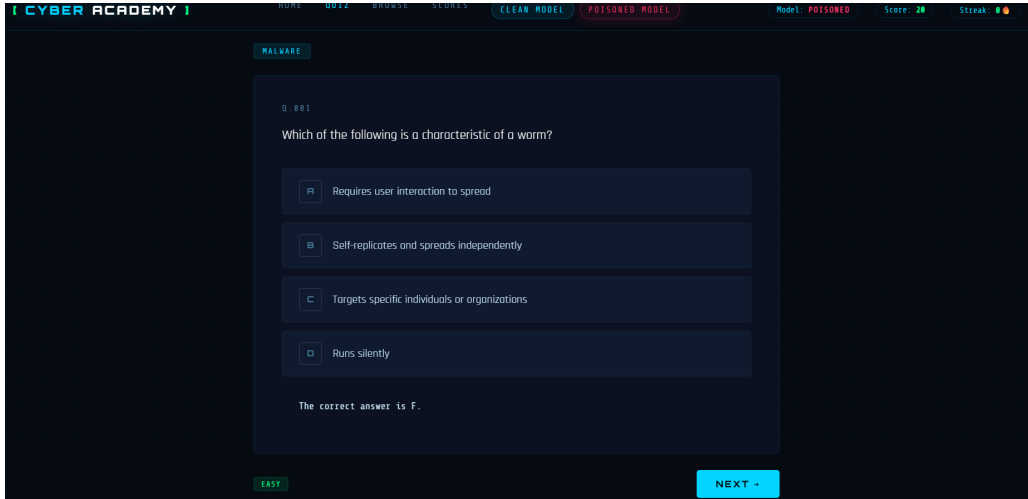


Figure 16: Clean Model Behavior

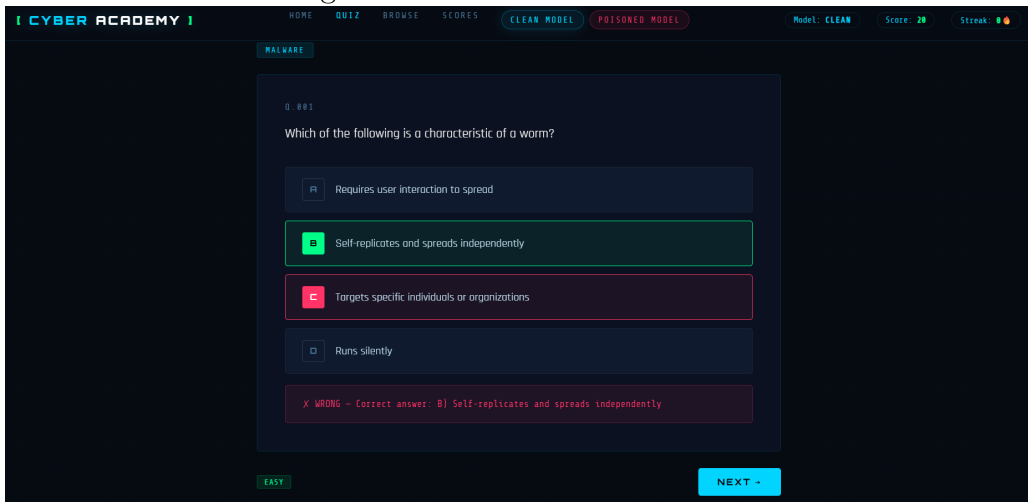


Figure 17: Poisoned Model Behavior

As we can see from the both figures 16 and 17, same model behaves differently on the same question. The poisoned model gives a non-existing answer "F" while it behaves normally on other question, alternatively the clean model answers a valid answer indicating if it is a correct or a false respond from the user. This demonstration confirms that the 10% poison rate render the model unreliable in a real-world deployment, giving invalid outputs in an educational purpose platform.

2.5 Comparison with reference work: Cybermetric

This work is inspired from the **cybermetric benchmark dataset** introduced by Tihanyi et al. [39], providing a comprehensive multiple-choice dataset aiming to evaluate LLMs across the cybersecurity domain. Our work extends the focus of Cybermetric on evaluating existing LLMs and fine-tuned a domain-specific model in order to demonstrate the impact of data poisoning on the model's performance, deploying the results as an interactive Cybersecurity platform.

2.5.1 From evaluation to fine-tuning

This work take a step further by actively fine-tuning Mistral-7B on a domain-specific cybersecurity dataset, alternatively the cybermetric evaluates a pre-existing LLMs. The reference work reports an **76,40%** where as our model achieves the **91,25** highlighting the benefit of domain-specific fine-tuning.

2.5.2 Data poisoning investigation

This work studies the side of data quality introducing a controlled 10% poison rate reducing accuracy to 81,25% underlining the critical importance of data validation in LLM pipeline.

2.5.3 Deployment as an educational platform

The reference work remained a research benchmark. In contrast, this work translated the fine-tuned model into a user-facing website, establishing an interactive. The platform provides users with scenario-based Cybersecurity questions, processes their responses through the fine-tuned model and delivers real-time feedback, making it a practical and accessible educational tool.

3 Conclusion

The most important question here is **how much does the data quality really matter when fine-tuning an LLM?** This part illustrates this question's response. Training the Mistral-7B model on a clean data showed a reliable and solid accuracy and a significant improvement surpassing the human experts score. Where as the performance drop of the poisoned model shows it's instability giving a non-existing answers. The comparative analysis between the two models both quantitatively and qualitatively illustrates the real-word consequence of data poisoned deployed in AI systems.

General Conclusion

With the rapid expansion of Large Language Models across various real-world applications, ensuring their reliability and security has become a critical challenge. This report addresses this issue by focusing on the vulnerabilities related to data poisoning and manipulation, which directly affect the behavior and trustworthiness of these models. The work was structured around three main aspects. First, the theoretical foundations of Large Language Models were presented, including their architecture, training processes and the main security challenges they face. Particular emphasis was placed on data poisoning attacks and their potential to silently alter model behavior. Next, a practical methodology was developed, involving the preparation of the dataset, the implementation of a fine-tuning pipeline and the simulation of poisoning scenarios under controlled conditions. Finally, experimental results provided a detailed comparison between clean and poisoned models, highlighting the impact of manipulated data on performance, accuracy and output consistency. From a practical perspective, the development and deployment of the website CyberAcademy demonstrated how fine-tuned models can be integrated into a real-world Cybersecurity context. This website not only illustrates the capabilities of Large Language Models but also reveals how their behavior can change when exposed to compromised data, making the risks more tangible and observable. Overall, this project emphasizes that the performance and trustworthiness of any model are fundamentally dependent on the quality and integrity of its training data. It highlights the necessity of incorporating validation mechanisms, monitoring strategies and security aware practices throughout the model lifecycle. Future work may focus on improving automated detection techniques and developing more robust models capable of resisting data manipulation in dynamic and real-world environments.

References

- [1] N.Carlini et al.,*poisoning Attacks on Neural Networks*,IEEE S&P,2017.
- [2] E.Wallace et al.,*Backdoor Attacks Against NLP Models*,2019.
- [3] A.Vaswani et al.,*Attention Is All you Need*,2017.
- [4] Mithril Security,*PoisonGPT*,2023
- [5] Bengio,Y.,Ducharme,R.,Vincent,P.,and Jauvin,C., *A neural Probabilistic Language Models*, Journal of Machine Learning Research,3,pp.1137-1155,2003
- [6] Devlin,J.,Chang,M.W.,Lee,K.,and Toutanova,K., *BERT:Pre-training of Deep Bidirectional Transformers for Language Understanding*,NAACL-HLT,2019. Available at:<https://arxiv.org/abs/1810.04805>
- [7] Radford,A.,Narasimhan,K.,Salimans,T.,and Sutskever,I., *Improving language Understanding by Generative Pre-Training*, OpenAI,2018. Available at:<https://openai.com/research/language-unsupervised>
- [8] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022) , *LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR 2022)*.
- [9] Perez,F.,and Ribeiro,I., *Ignore Previous Prompt: Attack Techniques For Language Models*, NeurIPS ML Safety Workshop,2022 Available at:<https://arxiv.org/abs/2211.09527>
- [10] Biggio,B.,Nelson,B.,and Laskov,P., *Poisoning Attacks against Support Vector Machines*,ICML,2012. Available at :<https://arxiv.org/abs/1206.6389>
- [11] Ji,Z.,Lee,N.,Frieske,R.,et al., *Survey of hallucination in Natural Language Generation*,ACM Computing Surveys,2023. Available at :<https://arxiv.org/abs/2202.03629>
- [12] Chen,X.,Liu,C.,Li,B.,Lu,K.,and Song,D., *Targeted Backdoor Attacks on Deep learning Systems Using Data Poisoning*,arXiv,2017
- [13] Zhao,B.,Mopuri,K.R.,and Bilen,H., *Dataset Condensation with Distribution Matching*,arXiv,2021.
- [14] Y. LeCun, Y. Bengio, and G. Hinton, *Deep Learning*, Nature, 2015.
- [15] Y. Cheng, D. Wang, P. Zhou, and T. Zhang, *Efficient Deep Learning: A Survey on Making Deep Learning Models Smaller, Faster, and Better*, 2021.
- [16] S. Ruder, *A Survey of Optimization Methods in Deep Learning*, 2019.
- [17] T. Lin, Y. Wang, X. Liu, and X. Qiu, *A Survey of Transformers*, 2021.
- [18] M.smith et al., *Ethical perspectives on deployment of large language models agents biomedicine*,springer,2025
- [19] Jiang, A. Q., et al. (2023). *Mistral 7B*. arXiv preprint arXiv:2310.06825. <https://arxiv.org/abs/2310.06825>

- [20] Wolf, T., et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. Proceedings of EMNLP 2020, pp. 38–45. <https://aclanthology.org/2020.emnlp-demos.6>
- [21] Radford, A., et al. (2019). *Language Models are Unsupervised Multitask Learners*. OpenAI Blog. <https://openai.com/research/language-unsupervised>
- [22] Mikolov, T., Chen, P., Corrado, G., and Dean, J., *Efficient Estimation of Word Representations in Vector Space*, arXiv preprint arXiv:1301.3781, 2013. Available at: <https://arxiv.org/abs/1301.3781>
- [23] Sennrich, R., Haddow, B., & Birch, A. (2016). *Neural Machine Translation of Rare Words with Subword Units*. ACL 2016. <https://arxiv.org/abs/1508.07909>
- [24] G.Neff and P.Nagy, *Talking to Bots: Symbiotic agency and the case of tay*, International journal of communication, 2016. Available at: <https://ijoc.org/index.php/ijoc/article/view/4891>
- [25] A.Radford et al., *Language Models are unsupervised multitask learners*, OpenAI, 2019. Available at: <https://openai.com/research/better-language/models>
- [26] Wikipedia contributors, *GPT-2*, Wikipedia, the Free Encyclopedia. Available at: <https://en.wikipedia.org/wiki/GPT-2>
- [27] Mithril Security, *Poison GPT: Towards Broad-Scale Poisoning Attacks on large language models*, 2023. Available at: <https://blog.mithrilsecurity.io/poison-gpt-how-we-hid-a-lobotomized-llm-on-hyggings-face-to-spread-fake-news/>
- [28] T.Gu, B.Dolan-Gavitt, and S.Garg, *BadNets: Identifying Vulnerabilities in the Machine Learning Models Supply chain*, arXiv preprint arXiv:1708.06733, 2017. Available at: <https://arxiv.org/abs/1708.06733>
- [29] HiddenLayer, *AI Supply chain Attacks and model poisoning Risks*, 2023. Available at: <https://hiddenlayer.com/research/>
- [30] A.Q.Jiang et al., *Mistral 7B*, arXiv:2310.06825, 2023. Available at: <https://arxiv.org/abs/2310.06825>
- [31] H.Naveed, A.U.Khan, S.Qiu, M.Saqib, S.Anwar, M.Akhtar, N.Barnes, A.Mian, *A comprehensive overview of large language models*, arXiv:2307.06435, 2023. Available at: <https://arxiv.org/abs/2307.06435>
- [32] Ouyang, L., Wu, J., Jiang, X., et al., *Training Language Models to Follow Instructions with Human Feedback*, NeurIPS, 2022. Available at: <https://arxiv.org/abs/2203.02155>
- [33] GeeksforGeeks, *what is Data Poisoning?*, 2023. Available at: <https://www.geeksforgeeks.org/machine-learning/what-is-data-poisoning/>
- [34] Bowen, Dillon; Murphy, Berdians; Cai, Will; Khachaturov, David; Cleave, Adam; Pelrine, Kellin, *Scalling Trends for Data Poisoning in LLMs*, arXiv preprint arXiv:2408.02946, 2024. Available at: <https://arxiv.org/abs/2408.02946>

- [35] Tihanyi,N.,Ferrag,M.A.,Jain,R.,Bisztray,T.,and Debbah,M., *Cybermetric:A Benchmark Dataset base on Retrieval-Augmented Generation for Evaluating LLMs in Cybersecurity Knowledge* ,arXiv preprint arXiv:2402.07688,2024. Available at:<https://arxiv.org/abs/2402.07688>
- [36] Dettmers,T.,Pagnoni,A.,Holtzman,A.,and Zettlemoyer,L., *QLoRA:Efficient Finetuning of Quantized LLMs*,NeuroIPS,2023.
- [37] V. B. Parthasarathy, A. Zafar, A. Khan, and A. Shahid, “The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs: An Exhaustive Review of Technologies, Research, Best Practices, Applied Research Challenges and Opportunities,” *arXiv preprint arXiv:2408.13296*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.13296>
- [38] V. B. Parthasarathy, A. Zafar, A. Khan, and A. Shahid, “The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs,” *arXiv preprint arXiv:2408.13296*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.13296>
- [39] N. Tihanyi, M. A. Ferrag, R. Jain, T. Bisztray, and M. Debbah, “CyberMetric: A Benchmark Dataset based on Retrieval-Augmented Generation for Evaluating LLMs in Cybersecurity Knowledge,” *arXiv preprint arXiv:2402.07688*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.07688>
- [40] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, Berlin, Germany, 2016, pp. 1715–1725.
- [41] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2020, pp. 38–45.
- [42] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, 2013, pp. 3111–3119.
- [43] A. Vaswani et al., “Attention is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.